

Adasa

Project Overview

Adasa is a single-page photography blog built with Angular. The application is written completely in Arabic and provides an elegant space where photographers can read, browse and learn about photographic techniques. It uses Angular's **standalone components** to avoid the overhead of `NgModule` definitions and is styled with **Bootstrap 5**. The blog content is stored in a local JSON file and loaded at runtime via an Angular service.

Purpose and Main Use Case

The goal of Adasa is to showcase photography articles in a clean, responsive interface. Visitors can:

- Explore featured and recent posts from the home page.
- Filter and search the blog by category or keywords.
- Read individual articles complete with a table of contents and recommended posts.
- Learn more about the authors and mission of the site on the "About" page.
- Review privacy and terms information.

The content focuses on teaching photographic skills to Arabic-speaking photographers.

□ Features

- **Standalone Angular architecture** – all components are standalone; the application is bootstrapped via `bootstrapApplication()` with providers for routing and HTTP ¹.
- **Responsive design with Bootstrap 5** – uses Bootstrap's grid and utilities for responsive layouts and custom CSS for hero sections and cards.
- **Home page with hero and statistics** – includes a hero section, featured posts, latest posts, category cards and a newsletter signup.
- **Blog listing page** – supports searching and filtering by category, toggling between grid and list views and pagination through a reusable pagination component.
- **Individual post page** – loads posts by slug, displays breadcrumbs, metadata, a dynamically generated table of contents and recommended posts from the same category.
- **About page** – describes the mission and values of Adasa and displays site statistics and author profiles.
- **Static policy pages** – Privacy and Terms pages explain data handling and usage rules.
- **404 handling** – a friendly 404 page is shown for unknown routes.

□ Tech Stack

Library / Tool	Version	Role
Angular	20.3.13	Core framework used with standalone components and Angular Router ¹ .
TypeScript	~5.9.3	Primary language for development and typings ² .
Bootstrap	5.3.8	CSS framework providing responsive grid, utility classes and components ² .
RxJS	~7.8.2	Reactive programming used in services to return observables ² .
FontAwesome Free	^7.1.0	Icon set used throughout the UI ² .
Jasmine & Karma	Angular CLI default	Unit testing framework (only a simple test is provided).

□ Project Structure

```
Adasa/  
├─ angular.json           // Angular CLI configuration  
├─ package.json           // Dependencies and scripts 2  
├─ src/  
│   └─ main.ts           // Bootstraps the application with standalone  
│                           components 1  
│   └─ index.html         // Host page  
│   └─ styles.css         // Global styles  
│   └─ assets/  
│       └─ posts.json     // Blog content (local JSON)  
│       └─ favicon.png    // Static assets  
│   └─ app/  
│       └─ app.component.ts // Root component  
│       └─ app.routes.ts   // Route definitions 3  
│       └─ layout/         // Layout components (navbar & footer)  
│       └─ pages/          // Feature pages (home, blog, post, about,  
│                           privacy, terms, not-found)  
│       └─ shared/         // Reusable UI components (pagination, post  
│                           card, section title)  
│       └─ services/       // Application services (e.g. PostsService) 4  
│       └─ models/        // Type definitions (e.g. Post interface)  
└─ public/  
    └─ favicon.ico
```

Main Directories

- **layout** – contains `navbar` and `footer` components used across all pages.
- **pages** – holds the primary views of the application (home, blog, individual post, about, privacy policy, terms of service and 404).
- **shared** – includes reusable UI components such as pagination controls, post cards and section titles.
- **services** – contains `PostsService` which loads post data from `assets/posts.json` ⁴.
- **models** – defines TypeScript interfaces (e.g. `Post`), ensuring strong typing throughout the app.

□ Setup & Run Instructions

1. Install dependencies

```
npm install
```

1. Run the development server

```
npm start  
# or  
ng serve
```

The application will start at `http://localhost:4200/` by default.

1. Run unit tests (optional)

```
ng test
```

□ Routing Overview

Path (Arabic)	Description	Component
<code>''</code>	Redirects to <code>الرئيسية/</code>	-
<code>/الرئيسية/</code>	Home page	<code>HomeComponent</code>
<code>/المدونة/</code>	Blog listing page with filters & search	<code>BlogComponent</code>
<code>/المدونة/:slug</code>	Blog post details by slug	<code>PostComponent</code>
<code>/من-نحن/</code>	About page	<code>AboutComponent</code>
<code>/سياسة-الخصوصية/</code>	Privacy policy	<code>PrivacyComponent</code>
<code>/شروط-الخدمة/</code>	Terms of service	<code>TermsComponent</code>
<code>**</code>	Wildcard route (404)	<code>NotFoundComponent</code>

Note: All routes are defined in `app.routes.ts` and configured via `provideRouter` ³.

□ Components Overview

Root & Layout

- **AppComponent** – root component bootstrapped by Angular; embeds the navbar at the top, a `RouterOutlet` for page content and the footer at the bottom.
- **NavbarComponent** – responsive navigation bar with links to the home, blog and about pages, a search icon and a call-to-action button. It includes a collapsible mobile menu.
- **FooterComponent** – footer with site description, social links, category lists computed from posts, a newsletter subscription form and links to privacy and terms pages.

Pages

- **HomeComponent** – landing page showing a hero section, site statistics, featured posts, category cards, latest posts and a newsletter signup form. It subscribes to `PostsService` to compute featured and latest posts.
- **BlogComponent** – lists all posts with support for category filtering, search, grid/list view toggling and pagination. Uses `PaginationComponent` for page navigation and tracks current page state in local variables.
- **PostComponent** – displays a single post based on the `slug` route parameter. It parses the markdown-like content into headings and paragraphs, builds a table of contents for quick navigation and lists up to three recommended posts from the same category. It unsubscribes from observables on destroy to avoid memory leaks.
- **AboutComponent** – describes the mission of Adasa, displays key site statistics (posts count, readers, categories and authors) and lists all unique authors from the post data.
- **PrivacyComponent** – static page outlining how user data is collected and used.
- **TermsComponent** – static page describing the terms of service.
- **NotFoundComponent** – simple 404 page shown when a route does not match.

Shared Components

- **PaginationComponent** – reusable pagination control that accepts total items, items per page and current page as inputs and emits page changes via an output event.
- **PostCardComponent** – card component for displaying a post in either grid or list layout (not used in the main blog view but available for reuse).
- **SectionTitleComponent** – simple component for rendering section headings and optional subtitles; used in static policy pages.

□ Services & Data

Adasa uses a single service:

- **PostsService** – provided in the root injector, this service loads blog posts from the local file `assets/posts.json` using the Fetch API and returns them as RxJS observables. It also provides a method to retrieve a single post by its `slug` ⁴. Since all data is local, there are no external API calls.

Data Flow

1. Components such as `HomeComponent`, `BlogComponent`, `PostComponent`, `AboutComponent` and `FooterComponent` subscribe to `PostsService.getPosts()` in their `ngOnInit` lifecycle hook.
2. The returned array of posts is filtered, sorted or mapped depending on the view (e.g. featured posts, latest posts, category counts).
3. When a post detail page is requested, `PostComponent` calls `getPostBySlug(slug)` and either displays the post or redirects back to the blog if the slug is invalid.
4. State such as current page, selected category and search term is stored in component properties; there is no global state management library.

□ UI & Styling

Bootstrap 5 provides the base styles and responsive grid system. Additional styling is defined in component-level `.css` files to implement gradients, cards, badges and animations. FontAwesome icons are used throughout the interface. The application is designed to be fully responsive across desktop and mobile devices.

□ Testing

The project includes a minimal unit test (`app.spec.ts`) using Jasmine and the Angular TestBed. The test suite verifies that the root component can be instantiated and that it renders the navbar, router outlet and footer. Tests can be executed with `ng test`, which runs Karma under the hood.

□ Strengths of the Project

- **Modern Angular architecture:** Adasa uses standalone components and `bootstrapApplication()` which simplifies module management and aligns with Angular's latest recommendations.
- **Well-structured layout:** The project separates layout, pages, shared components and services into clear directories, making it easy to understand and extend.
- **Responsive design:** Leveraging Bootstrap 5 and custom CSS ensures the site looks good on desktops and mobile devices.
- **Reactive data handling:** The use of RxJS observables for data retrieval and subscription patterns keeps data flows predictable and easy to manage.
- **Clean routing:** Routes are clearly defined and mapped to standalone components, with a wildcard for 404 handling ³.

¹ `main.ts`

file:///home/oai/share/Adasa/src/main.ts

² `package.json`

file:///home/oai/share/Adasa/package.json

3 **app.routes.ts**

file:///home/oai/share/Adasa/src/app/app.routes.ts

4 **posts.service.ts**

file:///home/oai/share/Adasa/src/app/services/posts.service.ts